

CS6510 · CODE WALK

Pipeline Analytics

Code Walk

How one accepted scan becomes a durable, deterministic popular-items ranking

Self-Checkout System · Pipeline Architecture

Pipeline where the data already flows

Checkout path

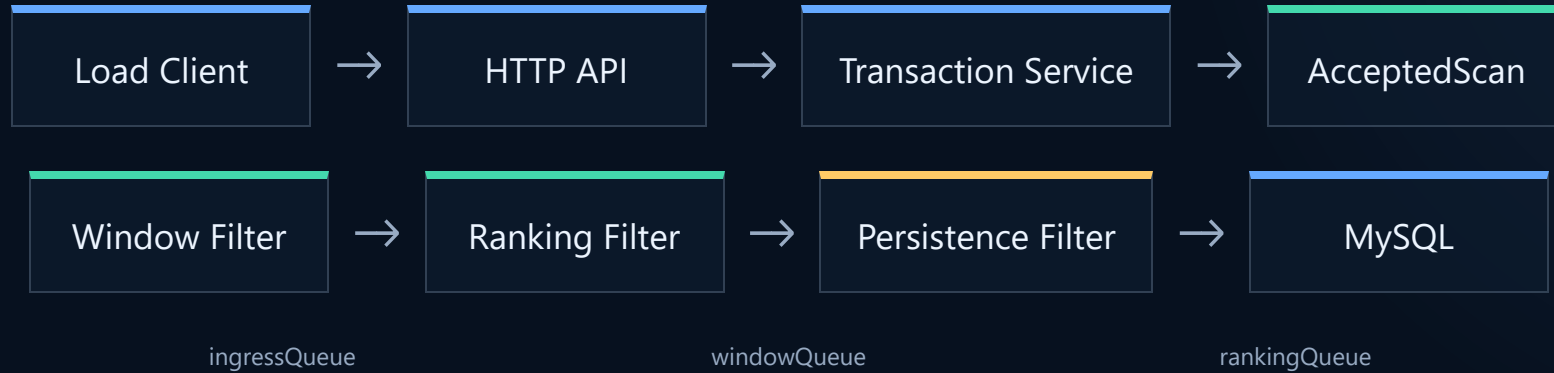
- Start a transaction
- Admit a basket scan
- Complete the purchase atomically
- Return a synchronous response

Analytics path

- Observe accepted scans
- Form fixed-size windows
- Rank the most scanned SKUs
- Persist completed rankings

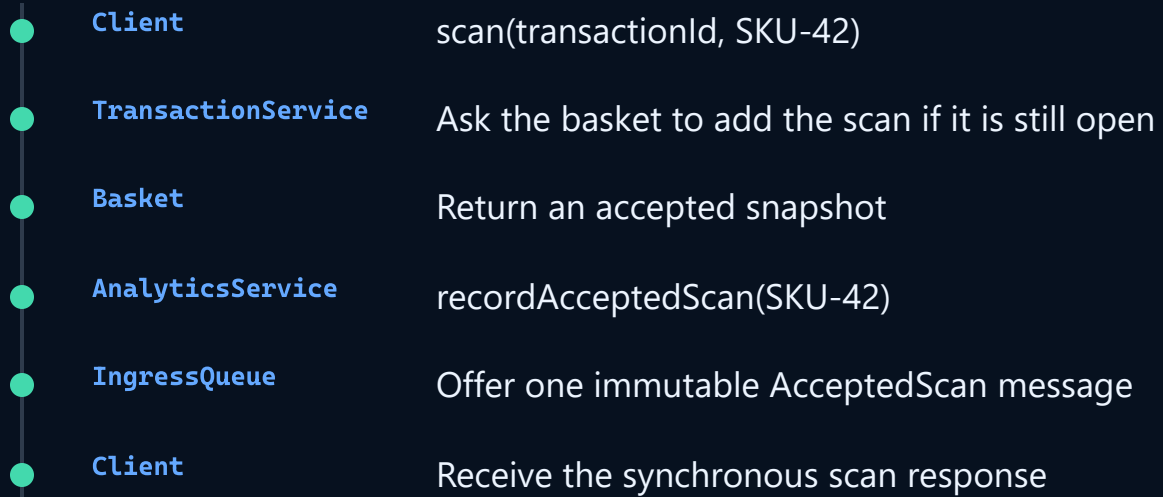
The public API remains synchronous. Only windowed analytics becomes a pipeline.

One synchronous boundary, three asynchronous stages



`Main` composes the graph and injects `analytics::recordAcceptedScan` into the transaction service.

A successful basket scan emits one analytics message



Admission happens first. An analytics outage cannot roll back an accepted customer scan.

Each filter performs one transformation

Window

Accepted scans
to immutable windows

queue

Ranking

Window snapshots
to deterministic Top 10

queue

Persistence

Ranked windows
to atomic DB commits

Isolation

Stages share messages, not mutable state

Ordering

FIFO queues preserve window order

Lifecycle

End markers drain every stage

A ring buffer forms exact 1,000-scan windows

1-500	501-1000	1001-1500	1501-2000
Emission		Window bounds	
Scan 1000		1-1000	
Scan 1500		501-1500	
Scan 2000		1001-2000	

The window overlaps by 500 scans, but storage remains fixed at 1,000 array slots.

```
1  static final int WINDOW_SIZE = 1_000;
2  static final int SLIDE_INTERVAL = 500;
3
4  static int ringIndex(long scanNumber) {
5      return (int) ((scanNumber - 1) % WINDOW_SIZE);
6  }
7
8  static boolean isEmissionEligible(long accepted) {
9      return accepted ≥ WINDOW_SIZE
10         && (accepted - WINDOW_SIZE)
11            % SLIDE_INTERVAL == 0;
12 }
```

Same window, same Top 10

Example window

```
1  A, B, A, C, A, B
```

Ranked output

```
1  1  A   3 scans
2  2  B   2 scans
3  3  C   1 scan
```

SKU ascending breaks count ties, so results stay reproducible across runs.

```
1  Map<String, Long> counts = new HashMap<>();
2  for (String sku : snapshot.skus())
3      counts.merge(sku, 1L, Long::sum);
4
5  List<Map.Entry<String, Long>> ordered =
6      new ArrayList<>(counts.entrySet());
7
8  ordered.sort(
9      Map.Entry.<String, Long>
10         comparingByValue(Comparator.reverseOrder())
11         .thenComparing(Map.Entry.comparingByKey()));
```

A ranking becomes visible only after one atomic commit



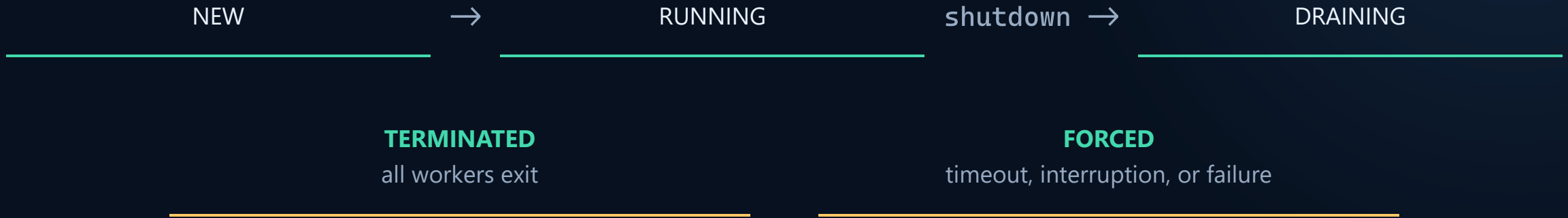
Reliability decisions

- One worker preserves FIFO ordering
- Header and ranks share one transaction
- Equivalent committed windows make retry safe
- Readers only observe committed snapshots

StoreFailure: retry after 50 ms, 200 ms, then 1 second per attempt.

A long database outage can grow the upstream queue.

End markers drain the pipeline in order



Normal shutdown

`IngressEnd` passes through Window, Ranking, and Persistence after earlier messages.

Abnormal shutdown

A worker failure marks the service failed; timeout or interruption forces worker termination.

Shutdown preserves order because the end marker travels through the same FIFO queues as the data.

The API returns the latest durable window

```
1 GET /analytics/popular-items?limit=10
2   ↓
3   ▼
4 AnalyticsHandler
5   ↓
6   ▼
7 AnalyticsService.latestPopularItems()
8   ↓
9   ▼
10 PopularWindowStore.readLatest()
```

The HTTP request never reads the live ring buffer.

```
1 {
2   "windowSize": 1000,
3   "slideInterval": 500,
4   "windowStart": 501,
5   "windowEnd": 1500,
6   "items": [
7     {
8       "sku": "SKU-000001",
9       "scanCount": 117,
10      "rank": 1
11    }
12  ]
13 }
```

The scan path stays stable under 10x station concurrency

Workload	Transactions/s	Items/s	Scan p99	Scan errors
10 stations · 60 s	153.5	2,144.3	0.43 ms	0%
100 stations · 120 s	159.4	3,698.4	0.46 ms	0%

999

no window yet

1000

emit 1-1000

1499

no new window

1500

emit 501-1500

Completion errors are a separate inventory outcome. Popular-items counts accepted scans, including baskets that may not later complete.

Pipeline fit and trade-offs

What worked

- Stage boundaries match the data transformations
- Queues isolate checkout latency from analytics work
- Immutable messages simplify concurrency
- Deterministic ranking supports repeatable tests
- Transactional storage prevents partial results

What comes next

- Bound or durably spool the ingress queue
- Publish queue depth and processing-lag metrics
- Add an explicit retry budget and alerting
- Maintain counts incrementally between windows
- Exercise recovery during a sustained DB outage

Pipeline architecture works best where data naturally crosses independent transformation stages.